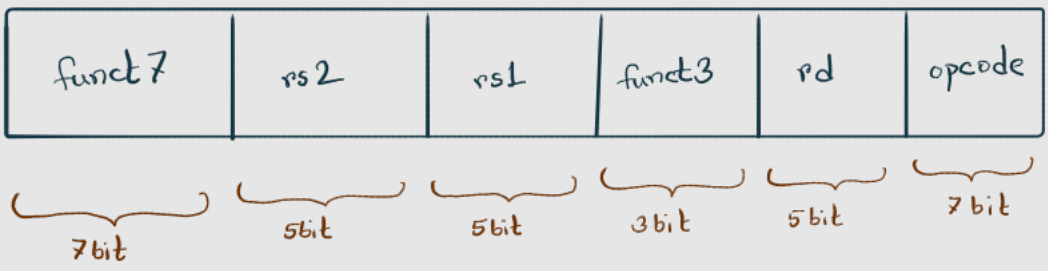
Translating a RISC-V assembly instruction into a Machine Instruction

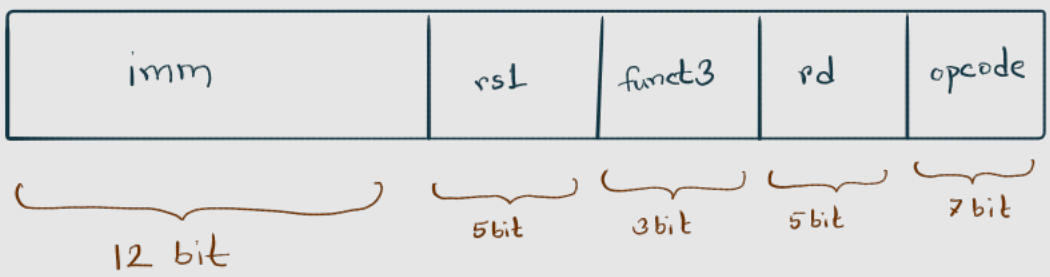
High Level Language
↓
Assembly
↓
Machine Language

Format:

R type:



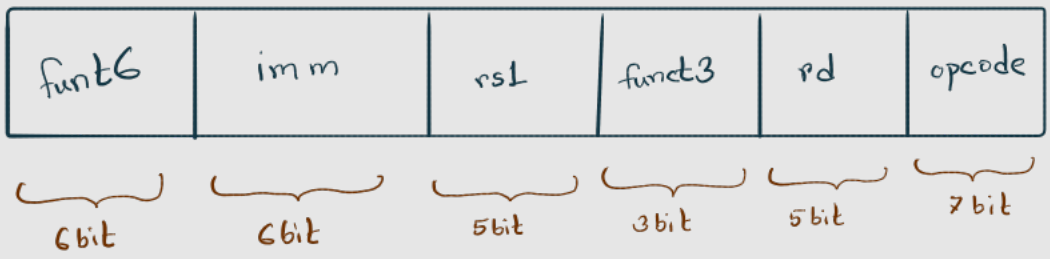
I type:



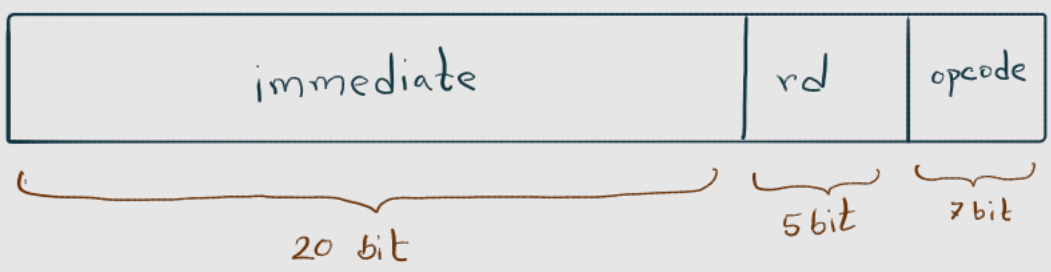
S type:



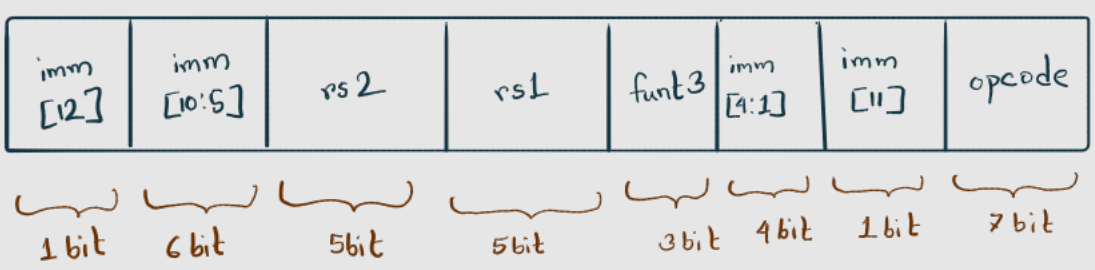
Shift (Similar to I type):



U Instruction:



SB type:



UJ type:



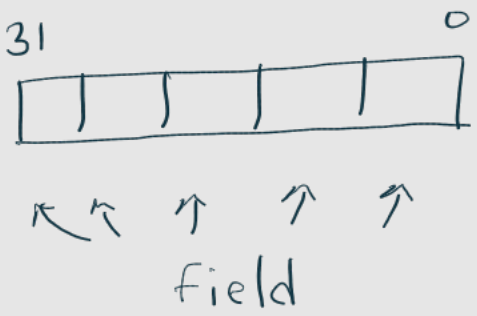
1. High-level program → **compiler** → assembly language program → **assembler** → machine language program.
2. Machine language only understands electric signals: high (1) and low (0).
3. A RISC-V instruction takes **exactly 32 bits** (one word). The layout is called the **instruction format**. We divide the 32 bits into **fields**.
4. **Conflict:** the desire to keep all instruction lengths the same, versus the desire to have a single instruction format.
5. **Design principle 3: Good design demands good compromise.** RISC-V keeps all instruction lengths the same, so it needs different instruction formats for different instructions.

Conflict:

Desire to keep all instruction same length

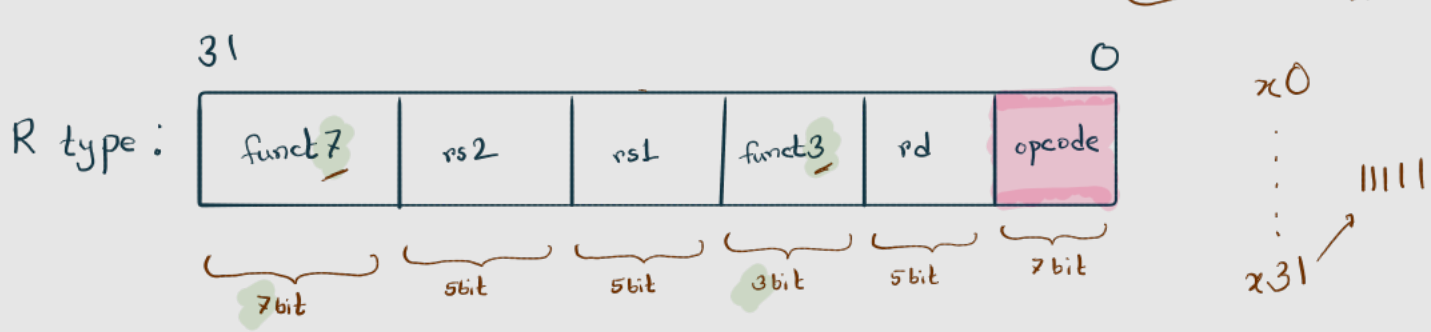
vs

Desire to keep a single instruction format (compromise)



① R Type instruction:
- 3 Register (2 rs and 1 rd)
- Example: add, sub, sll, XOR, AND, OR

add x5, x5, x6
rd rs1 rs2



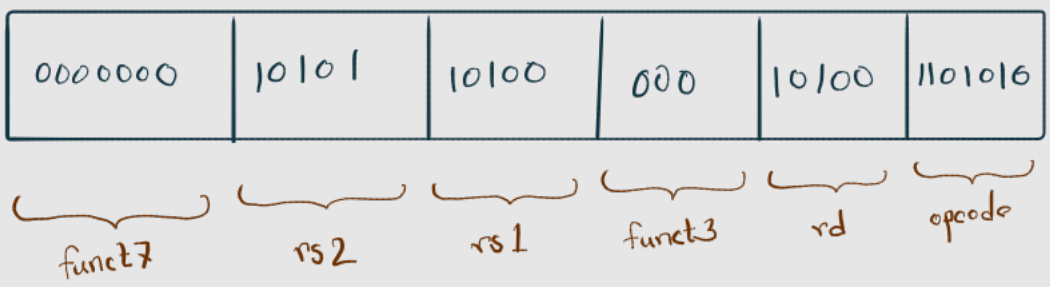
- Opcode, funct3, funct7 = Partially denotes the format of an instruction

add x20, x20, x21
rd rs1 rs2

(a) Encode
(b) Convert it into hex code

	opcode
R	1101010 ✓
I	1010101
S	1111001

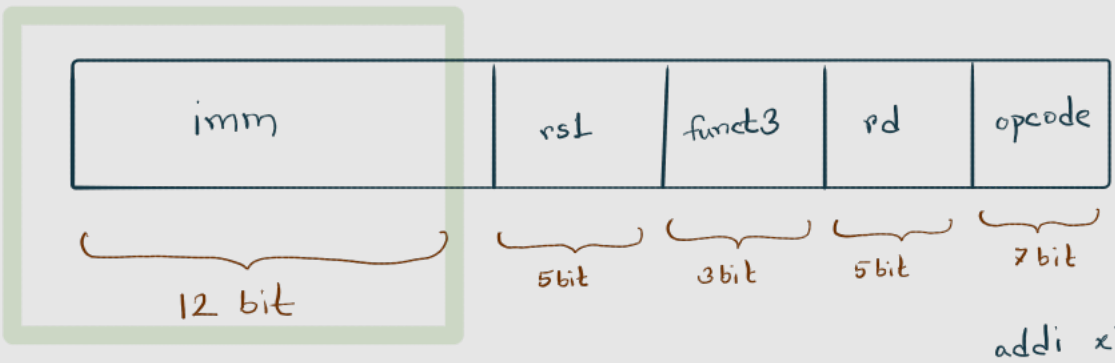
(a) R Type



(b) ← careful

0000	0001	0101	1010	0000	1010	0110	1010
= 0	1	5	A	0	A	6	A

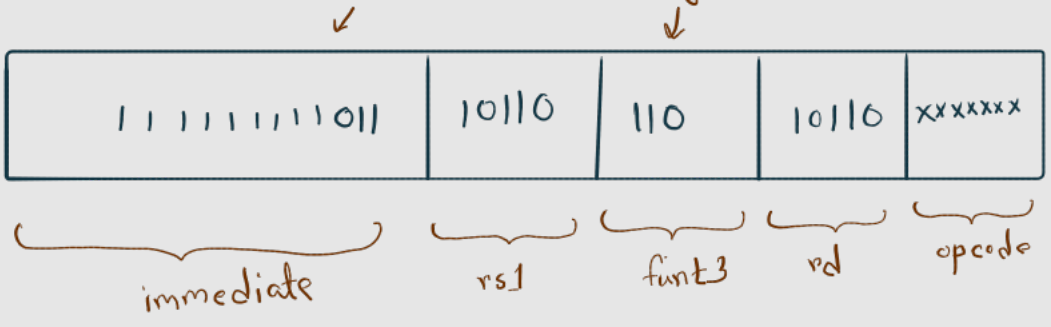
② I Type instruction : • 2 register and 1 immediate
• Ex: Addi, slli, ORI, ANDI, srli, Load ★



addi x5, x5, 1
rd rs1 imm/const

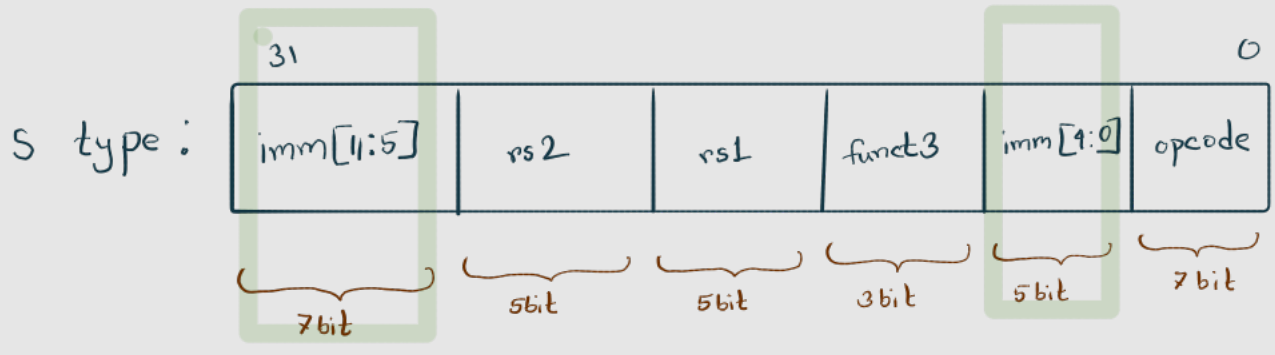
Immediate = Constant / offset
funct3 = Width + signedness ✓

addi x22, x22, -5 to binary code. 6 is the number used to explain width + signedness.
rs1
-5 6



(+5) = 101
(-5) = 011 [2s]

③ S Type instruction : * Ex: sd, sb, sh, sw



funct3: Represents size

sd x5, 16[x20]
rs2 rs1
off/imm

Theory : Why is the immediate split here?
Ans: • To keep rs1 and rs2 in the same place
• To keep the format similar

sd x5, 16[x20]. 010 represents sign

rs2 ↳ rs1, offset

000000000 10000

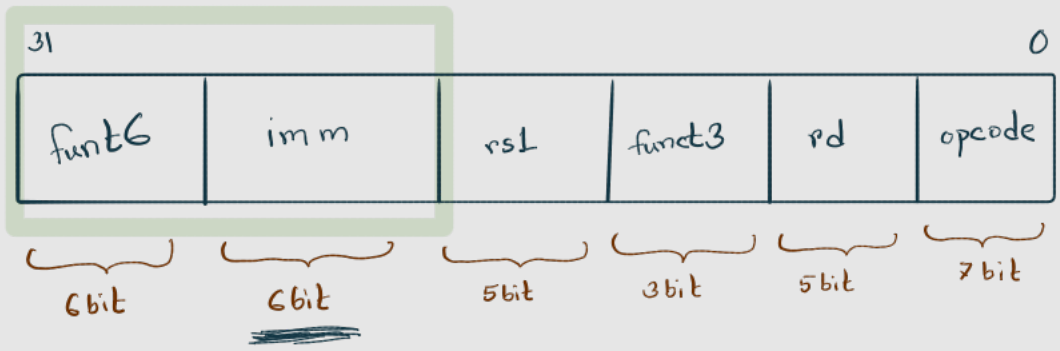


④ Shift Operation : Modified I type

slli x19, x20, 4

rd rs1 imm/const

Shift (Similar to I type) :



$2^6 = 64$

If we shift a value more than 63 times what will happen?

Ans: All becomes zero. Reason: imm has 6 bits. $2^6 = 64$

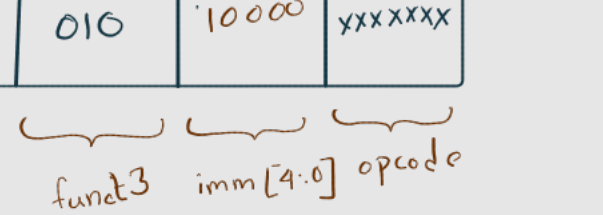
Forming a 32 bit immediate using

LUI

addi x5, x5, 2047 - [upto 12 bit]

max

imm



addi x5, x5, 4 000 000 [upto 32 bit]

LUI

Rule : * Copies the 20 bit data into rd's [31:12]

* Copies whatever you have in 31 bit to [63:32]

* Turn [11:0] into 0

Store $(0011\ 1101\ 0101\ 0000\ 0000)_2$ in x19 register

16 bit

[OR]

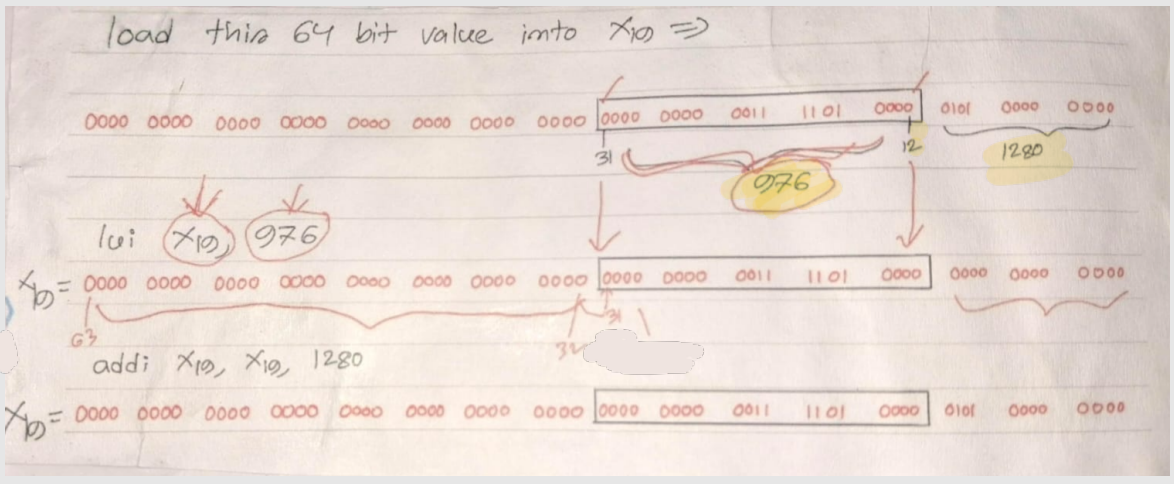
Store $(4000\ 000)_{10}$ in x19 register

LUI

Binary -> 12 bit

Decimal -> 2047

Hexadecimal -> 3



lui x19, 976

addi x19, x19, 1280

Write in assembly code / RISC-V code

int x = $(1000)_{10}$

Ans:

addi x10, x0, 1000

Write in assembly code / RISC-V code

int x = 6000

Ans: 6000 is bigger than 2047. We use lui

6000 = 1904

1 0 1 1 1 0 1 1 1 0 0 0 0

MSB LSB

lui x11, 1

addi x11, x11, 1904

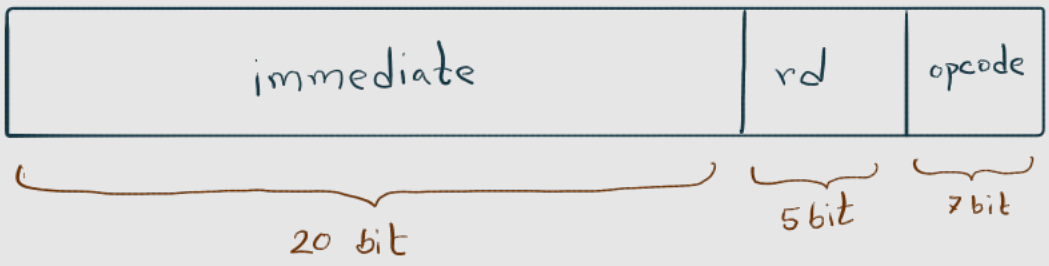
2 | 6000
2 | 3000 – 0 LSB
2 | 1500 – 0
2 | 750 – 0
2 | 375 – 0
2 | 187 – 1
2 | 93 – 1
2 | 46 – 1
2 | 23 – 0
2 | 11 – 1
2 | 5 – 1
2 | 2 – 1
2 | 1 – 0
0 – 1 MSB

int x = 0x12345083

0x → hexa

lui x5, 0x12345
addi x5, x5, 0x0835

U Instruction :



Construct the equivalent RISC-V code of the following C code:

```
int max (int a, int b){
    if(a>b) return a;
    else return b;
}

int main(){
    int x = max(1000, 6000);
}
```

Assume x is in x21.

x = x21
b = x10

```
main:
    addi x10, x0, 1000
    lui  x11, 1
    addi x11, x11, 1904

    jal  x1, max
    add  x21, x10, x0

max:
    bge  x11, x10, else
    jal  x0, return

else:
    addi x10, x11, 0

return:
    jalr x0, 0(x1)
```

lui

method call

store in x